



# ГОДИШЕН ПРОЕКТ

НА ТЕМА: ДЕСКТОП ПРИЛОЖЕНИЕ ЗА ИГРИ С ДУМИ -  
“БЕСЕНИЦА” И “WORDLE”

Изготвили:

Мирослава Мирославова Венелинова, 11.ж клас

Радослав Стефанов Цветков, 11.ж клас

Учител: Калоян Димитров

## СЪДЪРЖАНИЕ

|           |                                                |          |
|-----------|------------------------------------------------|----------|
| <b>1.</b> | <b>Въведение</b>                               | <b>4</b> |
| 1.1.      | Увод в обхвата на темата                       | 4        |
| 1.2.      | Цел на курсовата работа                        | 5        |
| 1.3.      | Задачи на курсовата работа                     | 5        |
| 1.4.      | Предназначение и целева група                  | 6        |
| <b>2.</b> | <b>Изложение</b>                               | <b>7</b> |
| 2.1.      | Обща структура на проекта                      | 7        |
| 2.1.1.    | Входна точка (Entry point)                     |          |
| 2.1.2.    | Потребителски интерфейс (UI layer)             |          |
|           | 2.1.2.1. Вход и навигация                      |          |
|           | 2.1.2.2. Игрален модул                         |          |
|           | 2.1.2.3. Администраторски модул                |          |
|           | 2.1.2.4. Модели и данни (Data layer & EF Core) |          |
| 2.2.      | Структура на базата данни                      | 10       |
| 2.2.1.    | Обща архитектура на базата и ER диаграма       |          |
| 2.2.2.    | Спецификация на таблиците                      |          |
| 2.2.3.    | Съхранени процедури                            |          |
| 2.2.4.    | Анализ на наличните данни (Data Overview)      |          |
| 2.2.5.    | Бъдещо оптимизиране на базата данни            |          |
| 2.3.      | Основни функционалности                        | 17       |
| 2.3.1.    | Start.cs (Вход в системата)                    |          |
| 2.3.1.1.  | Идентификация на потребителя (Вход)            |          |
| 2.3.1.2.  | Валидация с база данни                         |          |
| 2.3.2.    | ChooseGame.cs (Избор на игра)                  |          |
| 2.3.2.1.  | Показване на статистика                        |          |
| 2.3.2.2.  | Навигация                                      |          |
| 2.3.3.    | Admin.cs (FormAddNewWords - Админ панел)       |          |
| 2.3.3.1.  | Добавяне на думи в базата данни                |          |
| 2.3.4.    | Hangman.cs (FormHangman - Игра „Бесеница“)     |          |
| 2.3.4.1.  | Генериране на дума                             |          |
| 2.3.4.2.  | Инициализация на игралното поле                |          |
| 2.3.4.3.  | Визуализация на прогреса                       |          |
| 2.3.4.4.  | Обновяване на резултата                        |          |
| 2.3.5.    | Wordle.cs (FormWordle - Игра “Wordle”)         |          |
| 2.3.5.1.  | Потребителски интерфейс и виртуална клавиатура |          |
| 2.3.5.2.  | Инициализация и залагане на тайна дума         |          |
| 2.3.5.3.  | Следване на състоянието и статистика           |          |

|             |                                                                       |           |
|-------------|-----------------------------------------------------------------------|-----------|
| <b>2.4.</b> | <b>Взаимодействие потребител-приложение-база данни и UML-диаграма</b> | <b>19</b> |
| 2.4.1.      | Модел и управление на потребителите                                   |           |
| 2.4.2.      | Графичен потребителски интерфейс - GUI                                |           |
| 2.4.3.      | Достъп до данни                                                       |           |
| <b>2.5.</b> | <b>Използвани алгоритми</b>                                           | <b>21</b> |
| 2.5.1.      | Текстово съпоставяне с регулярни изрази (Regex)                       |           |
| 2.5.2.      | Генериране на псевдослучайни индекси (Randomization)                  |           |
| 2.5.3.      | Линейно търсене (Linear Search)                                       |           |
| 2.5.4.      | Динамично адресиране на GUI елементи (Рефлексия)                      |           |
| 2.5.5.      | Алгоритъм за оценка и оцветяване (Сравняване на низове)               |           |
| 2.5.6.      | Управление на състояния (State Machine чрез Switch-case)              |           |
| 2.5.7.      | Алгоритъм за псевдослучаен избор на тайна дума                        |           |
| 2.5.8.      | Алгоритъм за управление на състоянието на играта                      |           |
| <b>2.6.</b> | <b>Приложени принципи на ООП</b>                                      | <b>26</b> |
| 2.6.1.      | Капсулация (Encapsulation)                                            |           |
| 2.6.2.      | Наследяване (Inheritance)                                             |           |
| 2.6.3.      | Абстракция (Abstraction)                                              |           |
| 2.6.4.      | Използване на partial класове                                         |           |
| 2.6.5.      | Роля на EF Core в проекта и връзка с ООП                              |           |
| <b>2.7.</b> | <b>Етапи на разработка</b>                                            | <b>28</b> |
| 2.7.1.      | Анализ на изискванията и планиране.                                   |           |
| 2.7.2.      | Софтуерно проектиране и моделиране                                    |           |
| 2.7.3.      | Програмна реализация (Implementation / Coding)                        |           |
| 2.7.4.      | Тестване и отстраняване на грешки (Testing and Debugging)             |           |
| 2.7.5.      | Внедряване и документиране                                            |           |
| <b>2.8.</b> | <b>Използвани технологии</b>                                          | <b>30</b> |
| <b>2.9.</b> | <b>Роли на всеки от екипа</b>                                         | <b>31</b> |
| 2.9.1.      | Мирослава Венелинова                                                  |           |
| 2.9.2.      | Радослав Цветков                                                      |           |
| <b>3.</b>   | <b>Заклучение</b>                                                     | <b>32</b> |
| <b>4.</b>   | <b>Източници на информация</b>                                        | <b>33</b> |

## ВЪВЕДЕНИЕ

### 1. Увод в обхвата на темата

Напредъкът в технологиите прави софтуерните приложения неизменима част от нашето ежедневие, включително и в сферата на развлеченията. Класическите игри с думи развиват речниковият запас и логическото мислене, като тяхното дигитализиране и обединяване - конкретно “Бесеница” и “Wordle” - в едно общо приложение с интуитивен интерфейс, представлява актуален и практически полезен проект.

Още от много ранна възраст децата боравят с дигитални устройства, които използват за развлечение, като играене на игри, гледане на филми и социални мрежи. Вместо да си губят времето, може да им позволим да играят образователни игри, които както ще ги забавляват, така и ще им позволяват да научат нещо ново.

Игрите с думи развиват богат набор от когнитивни и комуникативни умения, включително **активен речников запас, бърза мисъл, памет, логика и концентрация**. Те тренират мозъка да прави бързи асоциации и подобряват изразяването.

Основните умения, които се развиват чрез тези игри, включват:

- **Езикови и комуникативни:** Разширяване на лексиката, научаване на нови думи и по-добро владение на граматиката.
- **Когнитивни:** Стимулират аналитичното мислене и подобряват фокуса, тъй като изискват спазване на правила и търсене на нестандартни решения.
- **Пространствени и зрителни:** Например при анаграми или редене на букви на дъска (като в Scrabble), мозъкът се упражнява да визуализира думите и тяхното разположение.
- **Социални:** Много от тези игри (думи по двойки, асоциации) насърчават екипната работа и активното слушане.

Доказано е, че редовното практикуване на такива занимания поддържа ума активен и подобрява паметта. Подобни упражнения са полезни за всички възрасти - при децата подпомагат грамотността, а при възрастните служат като превенция срещу когнитивни спадове.

Класическата игра „Бесеница“ е много повече от забавление - тя е мощен инструмент за развиване на редица когнитивни и езикови умения, като основните ползи от нея са:

- **Правопис и езикова грамотност:** Подобрява познанията за граматиката и помага за правилното изписване на думите.
- **Речников запас:** Разширява лексиката, тъй като играчът се сблъсква с нови термини и се учи да разпознава думи по контекст.
- **Логическо и дедуктивно мислене:** Участниците се учат да изключват грешни варианти и да използват логически стратегии (например, да търсят кои съгласни или гласни букви е най-вероятно да присъстват).

- **Концентрация и памет:** Изисква повишено внимание към детайлите и запомняне на вече използваните букви, за да не се повтарят грешки.

Игрите с думи от типа на Wordle също представляват отлична тренировка за мозъка, като те развиват следните ключови умения:

- **Логическо и дедуктивно мислене:** Използвате елиминация и подсказки (за точните и грешните букви), за да изградите следващия си ход.
- **Разширяване на речниковия запас:** Принуждавате се да се сещате за по-рядко използвани думи, съчетания от букви и техните структури.
- **Разпознаване на модели (Pattern recognition):** Мозъкът ви се учи да разпознава често срещани буквени комбинации, срички и окончания.
- **Работна памет и концентрация:** Трябва да помните кои букви вече сте използвали, къде се намират и кои опции сте изключили.
- **Устойчивост на стрес и търпение:** Играта ви учи да планирате ходовете си стратегически и да не действате прибързано.

## 2. Цели на проекта

Основните цели на проекта, който обединява в софтуерен продукт две игри с думи, които съчетават забавлението с интелектуално предизвикателство, могат да бъдат разделени в следните категории:

### Образователни и когнитивни цели

- Обогатяване на речта: Увеличаване на активния речников запас и запознаване с нови думи (синоними, антоними, фразеологизми).
- Подобряване на правописа: Затвърждаване на граматическите правила и правилното изписване.
- Развитие на паметта: Изискват запаметяване на букви, срички или вече използвани думи.

### Психологически и социални цели

- Намаляване на стреса: Действат разтоварващо и помагат за релаксация чрез превключване на вниманието

### Креативни и комуникативни цели

- Развитие на абстрактното мислене: Провокират участниците да правят асоциации и да разкриват скрити значения.

## 3. Задачи на проекта

Задачите, които си поставяме с този проект са:

- Да разработим функционалностите на десктоп приложението по начин, който да удовлетворява и изпълнява поставените цели за цялостния проект;
- Да имплементираме интуитивен интерфейс, който да позволява на потребителите да използват приложението, независимо от тяхната техническа грамотност;

- Да разработим завършен софтуерен продукт, който да удовлетворява потребността на потребителите да упражняват логическото си мислене;

#### **4. Предназначение и целева група**

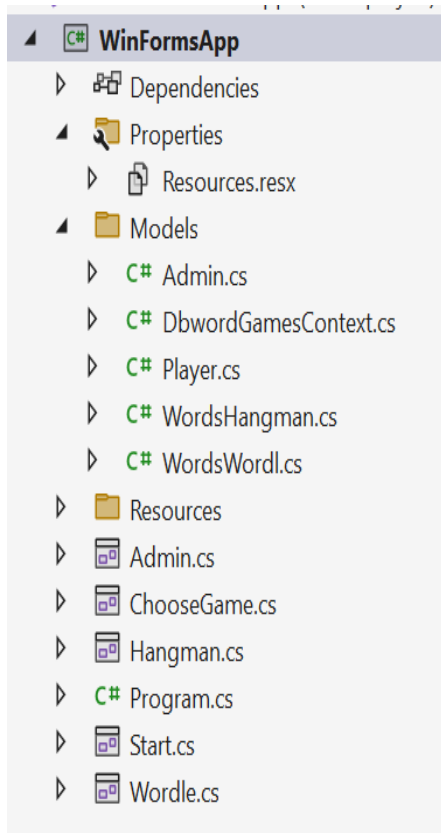
Проектът е предназначен за широка потребителска аудитория, която има желание да упражнява езиковите си умения на български език чрез играене на две от най-популярните игри - “Бесеница” и “Wordle”.

Десктоп приложението е подходящо за:

- Деца, като то подпомага ограмотяването, развива детската реч и прави ученето на нова лексика неусетно и приятно;
- Възрастни, като то служи като средство за умствена гимнастика (фитнес за мозъка), която доказано подобрява когнитивните функции, помага за превенция на мозъчното стареене;

## ИЗЛОЖЕНИЕ

### 1. Обща структура на проекта



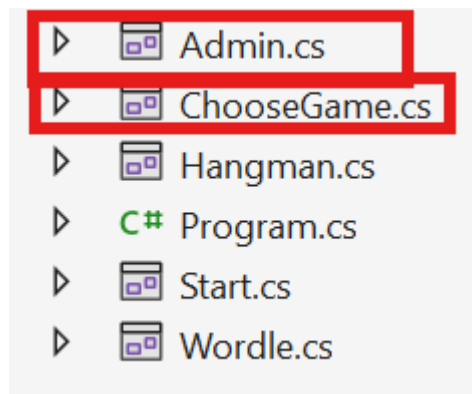
#### 1.1. Входна точка (Entry Point) -

Program.cs, съдържа главния метод на приложението (Main()). Той инициализира системните конфигурации и стартира първоначалния прозорец:

```
static void Main()
{
    ApplicationConfiguration.Initialize();
    Application.Run(new Start());
}
```

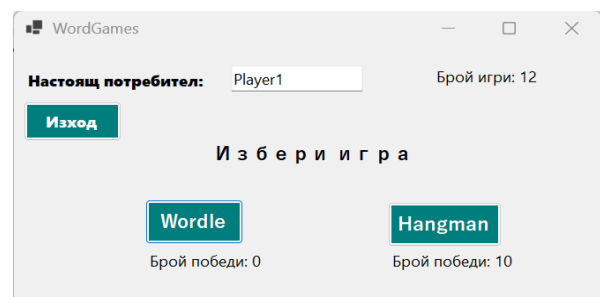
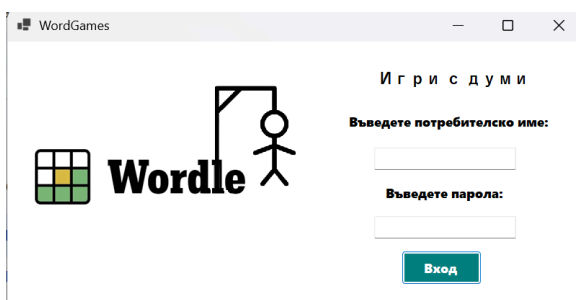
#### 1.2. Потребителски интерфейс (UI Layer)

- приложението е разделено на две основни части, съобразени с ролята на използващия го (за потребители и за администратори), управлявани от следните форми:



##### 1.2.1. Вход и навигация

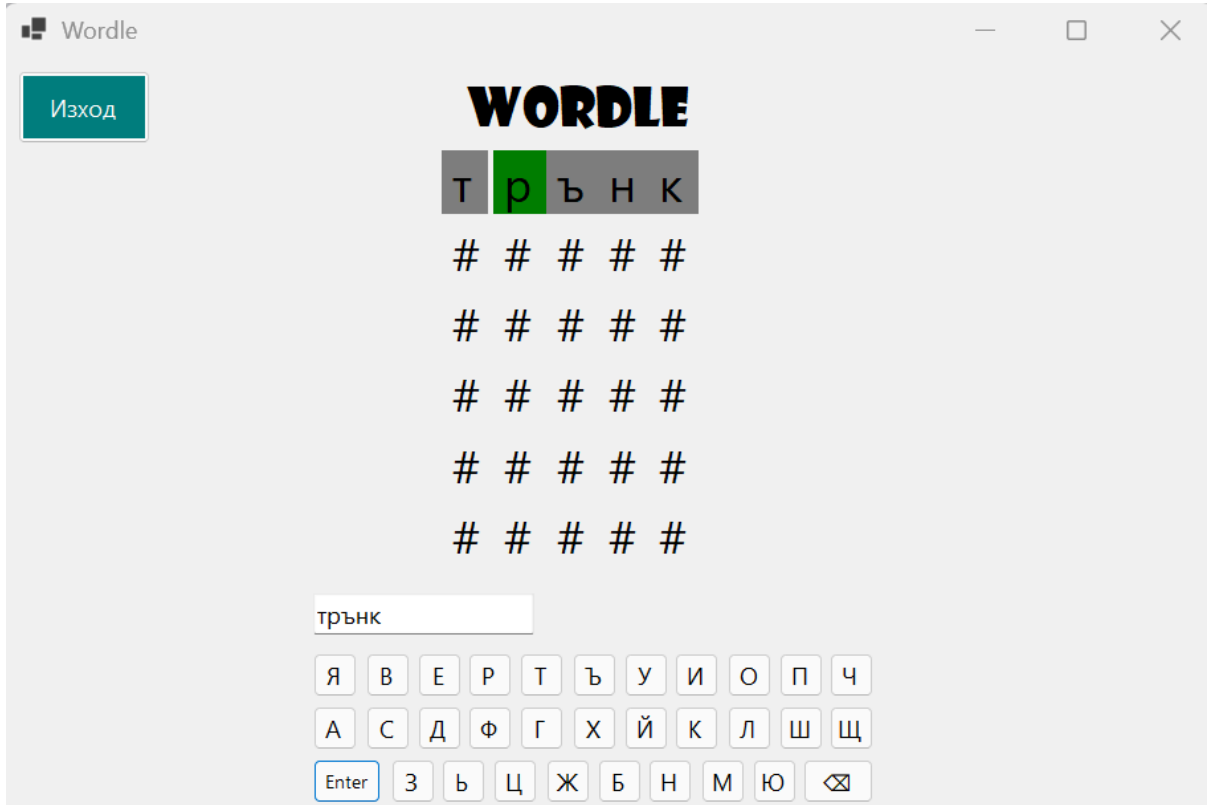
- *Start.cs* - Екран за автентикация - проверява паролата и разделя потребителите според ролята им. Ако името отговаря на админ шаблон (#A[1-5]), пренасочва към администраторския панел, в противен случай — към този за избора на игра, т.е. потребителският панел;



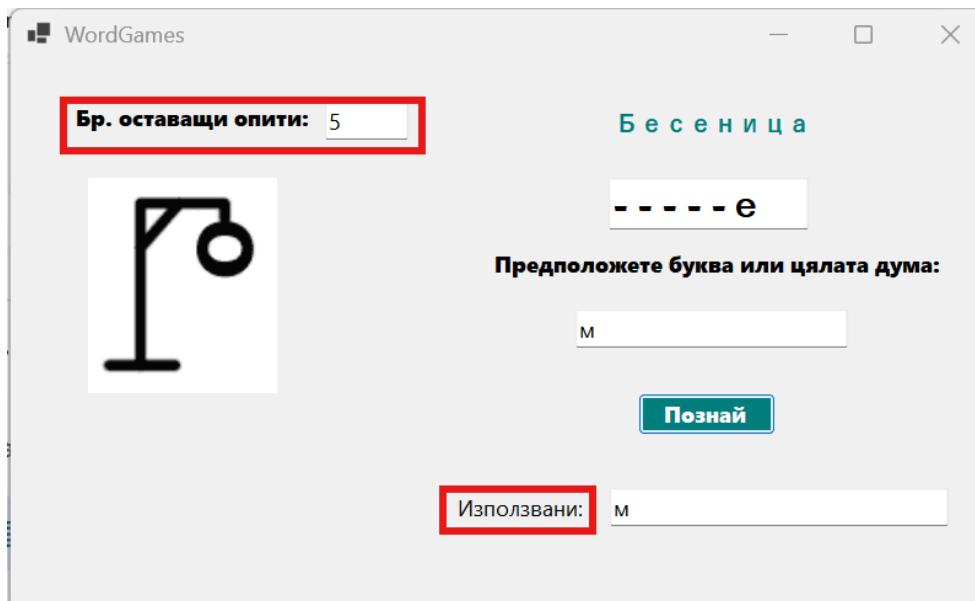
- *ChooseGame.cs* - зарежда статистиката на потребителя и служи като разпределител към игрите.

### 1.2.2. Игрален модул (Игри)

- *Wordle.cs* - реализира играта "Wordle" (5-буквени думи, които трябва да се познаят в рамките на 6 опита) с изградена виртуална клавиатура на български език и динамична матрица от полета за проверка за съвпадение на символите, съдържащи се в скритата дума и потребителското предположение;



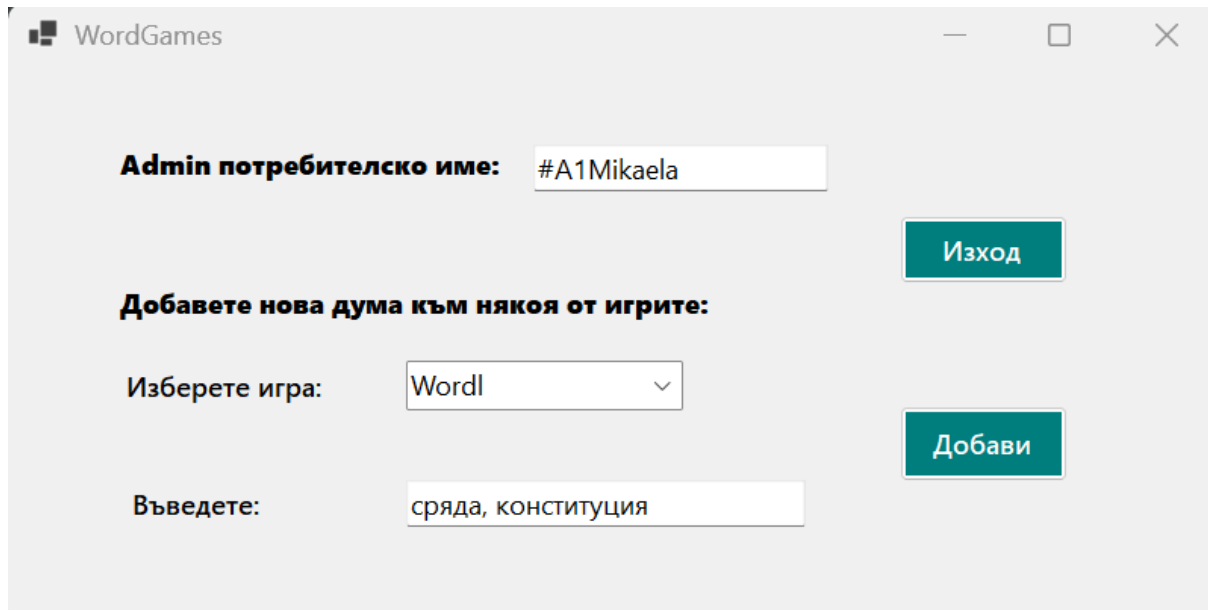
- *Hangman.cs* - реализира класическата игра "Бесеница", като има визуален брояч, представящ оставащите опити на играча, списък с натрупване на сгрешени букви, т.е. символи, които вече са били използвани като част от предположението и графична промяна на състоянието, която чрез алгоритъм визуализира разкриването на букви от думата, в случай на правилно въвеждане.





### 1.2.3. Администраторски модул

- *Admin.cs* (FormAddNewWords) - представлява специализиран панел за оторизирани администраторски профили, който позволява въвеждане на голям набор от нови думи в базата данни за двете игри.



### 1.2.4. Модели и данни (Data layer & EF Core)

- *DbwordGamesContext* - главният клас (контекстът на EF Core), отварящ и управляващ сесията към SQL базата данни;

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace WinFormsApp.Models;

13 references
public partial class DbwordGamesContext : DbContext
{
    5 references
    public DbwordGamesContext()
    {
    }

    0 references
    public DbwordGamesContext(DbContextOptions<DbwordGamesContext> options)
        : base(options)
    {
    }

    1 reference
    public virtual DbSet<Admin> Admins { get; set; }

    1 reference
    public virtual DbSet<Player> Players { get; set; }

    4 references
    public virtual DbSet<WordsHangman> WordsHangmen { get; set; }

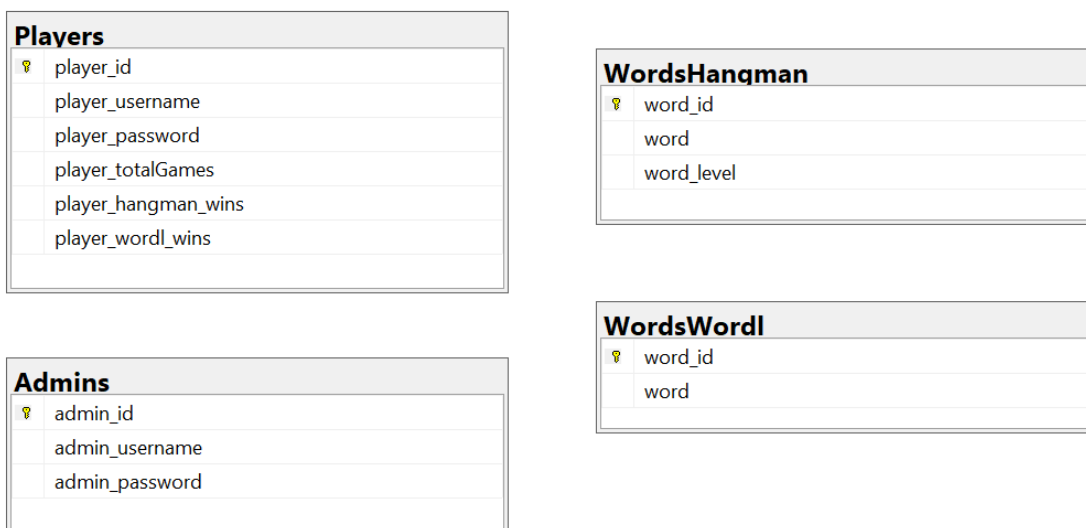
    2 references
    public virtual DbSet<WordsWordl> WordsWordls { get; set; }
```

- *Таблицы/Субекти (Entities)* - реализирани като virtual DbSet<T> - позволява се лесно тестване на единици с библиотеки за симулиране и улеснява динамичното генериране на прокси за функции като отложено зареждане.

## 2. Структура на базата данни

### 2.1. Обща архитектура на базата и ER диаграма

Базата данни се състои от 4 основни таблици. Към момента схемата е децентрализирана - таблиците нямат дефинирани външни ключове (Foreign Keys) помежду си, което означава, че логиката за свързване на данни (например кои думи са играни от даден играч) се управлява на приложно ниво.



#### Физически модел (Таблицы):

- **Players**: Потребителските профили и статистика за изиграните игри и победи;
- **Admins**: Пази данни за администраторите на системата;
- **WordsHangman**: Речник с думи, разпределени по трудности за „Бесеница“;
- **WordsWordl**: Речник с 5-буквени думи за играта „Wordle“.

### 2.2. Спецификация на таблиците

#### 2.2.1. Таблица Players

Съхранява информация за регистрираните играчи и тяхната персонална гейминг статистика.

| Име на колона                | Тип данни    | Ограничения (Constraints)  | Описание                           |
|------------------------------|--------------|----------------------------|------------------------------------|
| <code>player_id</code>       | INT          | IDENTITY(1,1), PRIMARY KEY | Уникален идентификатор на играча.  |
| <code>player_username</code> | NVARCHAR(20) | NOT NULL                   | Потребителско име (до 20 символа). |

|                     |              |          |                                        |
|---------------------|--------------|----------|----------------------------------------|
| player_password     | NVARCHAR(20) | NOT NULL | Парола за достъп (до 20 символа).      |
| player_totalGames   | INT          | NOT NULL | Общ брой изиграни игри от потребителя. |
| player_hangman_wins | INT          | NOT NULL | Брой победи на играта „Бесеница“.      |
| player_wordl_wins   | INT          | NOT NULL | Брой победи на играта „Wordle“.        |

Програмна имплементация на таблицата Players, генерирана автоматично от EF Core Framework, като реализацията е с partial class:

```
namespace WinFormsApp.Models;

8 references
public partial class Player
{
    2 references
    public int PlayerId { get; set; }

    3 references
    public string PlayerUsername { get; set; } = null!;

    2 references
    public string PlayerPassword { get; set; } = null!;

    7 references
    public int PlayerTotalGames { get; set; }

    4 references
    public int PlayerHangmanWins { get; set; }

    3 references
    public int PlayerWordlWins { get; set; }
}
```

### 2.2.2. Таблица Admins

Съхранява данни за администраторите, които имат права да управляват съдържанието (например да добавят нови думи).

| Име на колона  | Тип данни    | Ограничения                   | Описание                                  |
|----------------|--------------|-------------------------------|-------------------------------------------|
| admin_id       | INT          | IDENTITY(1,1),<br>PRIMARY KEY | Уникален идентификатор на администратора. |
| admin_username | NVARCHAR(20) | NOT NULL                      | Администраторско потребителско име.       |
| admin_password | NVARCHAR(20) | NOT NULL                      | Парола на администратора.                 |

Програмна имплементация на таблицата Admins генерирана автоматично от EF Core Framework, като реализацията е с partial class:

```
namespace WinFormsApp.Models;
```

3 references

```
public partial class Admin
```

```
{
```

2 references

```
public int AdminId { get; set; }
```

3 references

```
public string AdminUsername { get; set; } = null!;
```

2 references

```
public string AdminPassword { get; set; } = null!;
```

```
}
```

### 2.2.3. Таблица WordsHangman

Съдържа списък с думи, използвани за играта „Бесеница“ и тяхната сложност.

| Име на колона | Тип данни | Ограничения                   | Описание                          |
|---------------|-----------|-------------------------------|-----------------------------------|
| word_id       | INT       | IDENTITY(1,1),<br>PRIMARY KEY | Уникален идентификатор на думата. |

|            |               |          |                                                         |
|------------|---------------|----------|---------------------------------------------------------|
| word       | NVARCHAR(100) | NOT NULL | Самата дума (поддържа Unicode/Кирилица до 100 символа). |
| word_level | NVARCHAR(10)  | NOT NULL | Ниво на трудност (напр. 'лесна', 'средна', 'сложна').   |

Програмна имплементация на таблицата WordsHangman, генерирана автоматично от EF Core Framework, като реализацията е с partial class:

```
namespace WinFormsApp.Models;

6 references
public partial class WordsHangman
{
    3 references
    public int WordId { get; set; }

    11 references
    public string Word { get; set; } = null!;

    1 reference
    public string WordLevel { get; set; } = null!;
}
```

#### 2.2.4. Таблица WordsWordl

Речник с думи за играта „Wordle“. Тъй като играта стандартно изисква 5-буквени думи, размерът на полето е оптимизиран.

| Име на колона | Тип данни    | Ограничения                   | Описание                          |
|---------------|--------------|-------------------------------|-----------------------------------|
| word_id       | INT          | IDENTITY(1,1),<br>PRIMARY KEY | Уникален идентификатор на думата. |
| word          | NVARCHAR(10) | NOT NULL                      | Думата за Wordle (на Кирилица).   |

Програмна имплементация на таблицата WordsWordle, генерирана автоматично от EF Core Framework, като реализацията е с partial class:

```
namespace WinFormsApp.Models;

8 references
public partial class WordsWordl
{
    3 references
    public int WordId { get; set; }

    96 references
    public string Word { get; set; } = null!;
}
```

### 2.3. Съхранени процедури

За по-добра сигурност и капсулация на данните, вмъкването на записи се извършва изцяло чрез съхранени процедури (Stored Procedures).

usp\_AddPlayer - Добавя нов играч в системата с първоначална статистика.

- Параметри:
  - @player\_username NVARCHAR(20)
  - @player\_password NVARCHAR(20)
  - @player\_total\_games INT
  - @player\_hangman\_wins INT
  - @player\_wordl\_wins INT

```
1  CREATE OR ALTER PROC usp_AddPlayer
2      @player_username NVARCHAR(20), @player_password NVARCHAR(20),
3      @player_total_games INT, @player_hangman_wins INT, @player_wordl_wins INT
4  AS
5  INSERT INTO Players
6      VALUES(@player_username, @player_password, @player_total_games,
7              @player_hangman_wins, @player_wordl_wins)
8  GO
```

usp\_AddAdmin - Създава нов администраторски профил.

- Параметри:
  - @admin\_username NVARCHAR(20)
  - @admin\_password NVARCHAR(20)

```
10 CREATE OR ALTER PROC usp_AddAdmin
11     @admin_username NVARCHAR(20), @admin_password NVARCHAR(20)
12 AS
13 INSERT INTO Admins
14     VALUES(@admin_username, @admin_password)
15 GO
```

usp\_AddWordHangman - Добавя нова дума в речника за „Бесеница“.

- Параметри:
  - @word NVARCHAR(100)
  - @word\_level NVARCHAR(20)

```
17  CREATE OR ALTER PROC usp_AddWordHangman
18      @word NVARCHAR(100), @word_level NVARCHAR(20)
19  AS
20  INSERT INTO WordsHangman
21  VALUES(@word, @word_level)
22  GO
```

usp\_AddWordWordl - Добавя нова дума в речника за „Wordle“.

- Параметри:
  - @word NVARCHAR(10)

```
24  CREATE OR ALTER PROC usp_AddWordWordl
25      @word NVARCHAR(10)
26  AS
27  INSERT INTO WordsWordl
28  VALUES(@word)
29  GO
```

#### 2.4. Анализ на наличните данни (Data Overview)

В базата данни първоначално са заредени тестови данни, които демонстрират разпределението на потребителите и игровото съдържание:

##### 2.4.1. Профили и статистика на играчите

Въведени са 10 играчи с различни нива на активност. От данните се вижда ясно профилирането на потребителите:

- 2.4.1.1. WordlEnthusiast е най-активният Wordle играч с 13 победи;
- 2.4.1.2. Player1 е най-успешният Hangman играч с 10 победи;
- 2.4.1.3. Има и напълно нови потребители без изиграни игри.

##### 2.4.2. Администратори

Въведени са 5 администраторски акаунта (#A1Mikaela до #A5Stoyan), чиито пароли следват ясна номенклатурна структура (напр. admins1MI).

##### 2.4.3. Лингвистично съдържание (Речници)

- WordsHangman (40 думи): Думите са на български език и са класифицирани в три нива на трудност:
  - Лесна: Къси или стандартни думи (напр. „котка“, „стол“, „влак“, „дим“).

- Средна: По-дълги думи или такива с по-специфична структура (напр. „спокоен“, „площад“, „слънце“).
- Сложна: Дълги думи или думи, съдържащи по-рядко срещани букви като „щ“, „я“, „ъ“ (напр. „мръсен“, „любопитен“, „велосипед“, „безразличен“, „небрежен“).
- WordsWordl (62 думи): Всички думи са точно 5-буквени и са на български език, съобразно правилата на играта Wordle (напр. „какво“, „добре“, „време“, „книга“, „песен“, „лекар“, „шефът“).

## 2.5. Бъдещо оптимизиране на базата данни

- **Сигурност на паролите:** В момента паролите (player\_password и admin\_password) се съхраняват в чист текст (NVARCHAR). Изключително важно е да се премине към хеширане на паролите (напр. с HASHBYTES в SQL Server или чрез външна библиотека на приложно ниво) и увеличаване на размера на полето до поне 64 символа.
- **Ограничения за уникалност (Unique Constraints):** Трябва да се добави UNIQUE ограничение за колоните player\_username и admin\_username, за да се предотврати регистрацията на двама потребители с еднакви имена.
- **Индексиране (Indexing):** Препоръчително е да се създадат индекси (Non-Clustered Indexes) върху търсените полета, като player\_username, word и word\_level, за по-бързо бързодействие при проверка на входни данни и селектиране на произволни думи за игра.
- **Нормализация и релации:** За да се пази история на игрите, може да се създаде нова таблица GamesHistory(game\_id, player\_id, game\_type, is\_won, played\_at), която ще има връзка (Foreign Key) към таблицата Players.



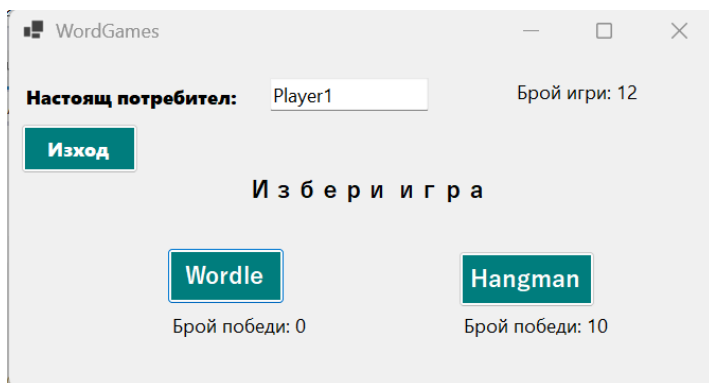
### 3. Основни функционалности

#### 3.1. Start.cs (Вход в системата)

**3.1.1. Идентификация на потребителя (Вход):** разделя потребителите на два типа въз основа на потребителското име. Използва се регулярен израз за проверка дали влиза администратор;

**3.1.2. Валидация с база данни:** извършва се заявка към контекста на базата данни (DbwordGamesContext), за да се намери съвпадение за потребителско име и парола. При успех приложението пренасочва към съответния екран (FormAddNewWords или FormChooseGame).

#### 3.2. ChooseGame.cs (Избор на игра)



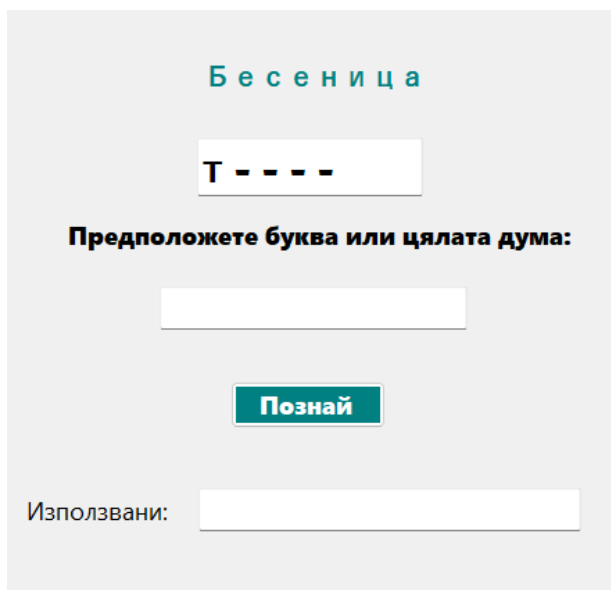
**3.2.1. Показване на статистика:** при зареждане на формата се визуализира потребителското име, общият брой изиграни игри и броят победи за двете налични игри (Wordle и Hangman);

**3.2.2. Навигация:** дава възможност за стартиране на една от двете игри или за изход от профила.

#### 3.3. Admin.cs (FormAddNewWords - Админ панел)

**3.3.1. Добавяне на думи в базата данни:** Администраторът може да въвежда списък от думи в текстово поле, отделени чрез разделител, като в зависимост от избраната игра в падащото меню (comboBoxGames), думите се записват в таблицата за Wordle или за Бесеница.

#### 3.4. Hangman.cs (FormHangman - Игра „Бесеница“)



**3.4.1. Генериране на дума:** при стартиране се избира произволна дума от базата данни.

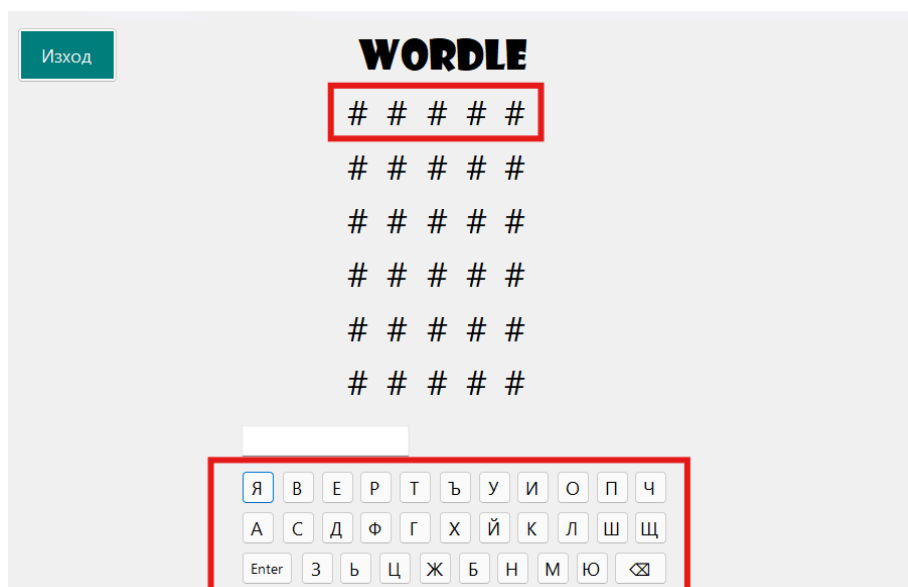
**3.4.2. Инициализация на игралното поле:** скрива думата с тирета (-), като на случаен принцип разкрива една буква за улеснение на играча.

**3.4.3. Визуализация на прогреса:** при грешен опит броят на оставащите опити намалява и се променя изображението на бесеницата (ChangePicture()). Използваните грешни букви се визуализират в отделно поле.

- 3.4.4. Обновяване на резултата:** При победа или загуба се актуализира статистиката на текущия играч (`PlayerTotalGames`, `PlayerHangmanWins`) и се отваря екранът за избор на игра.

### 3.5. Wordle.cs (FormWordle - Игра “Wordle”)

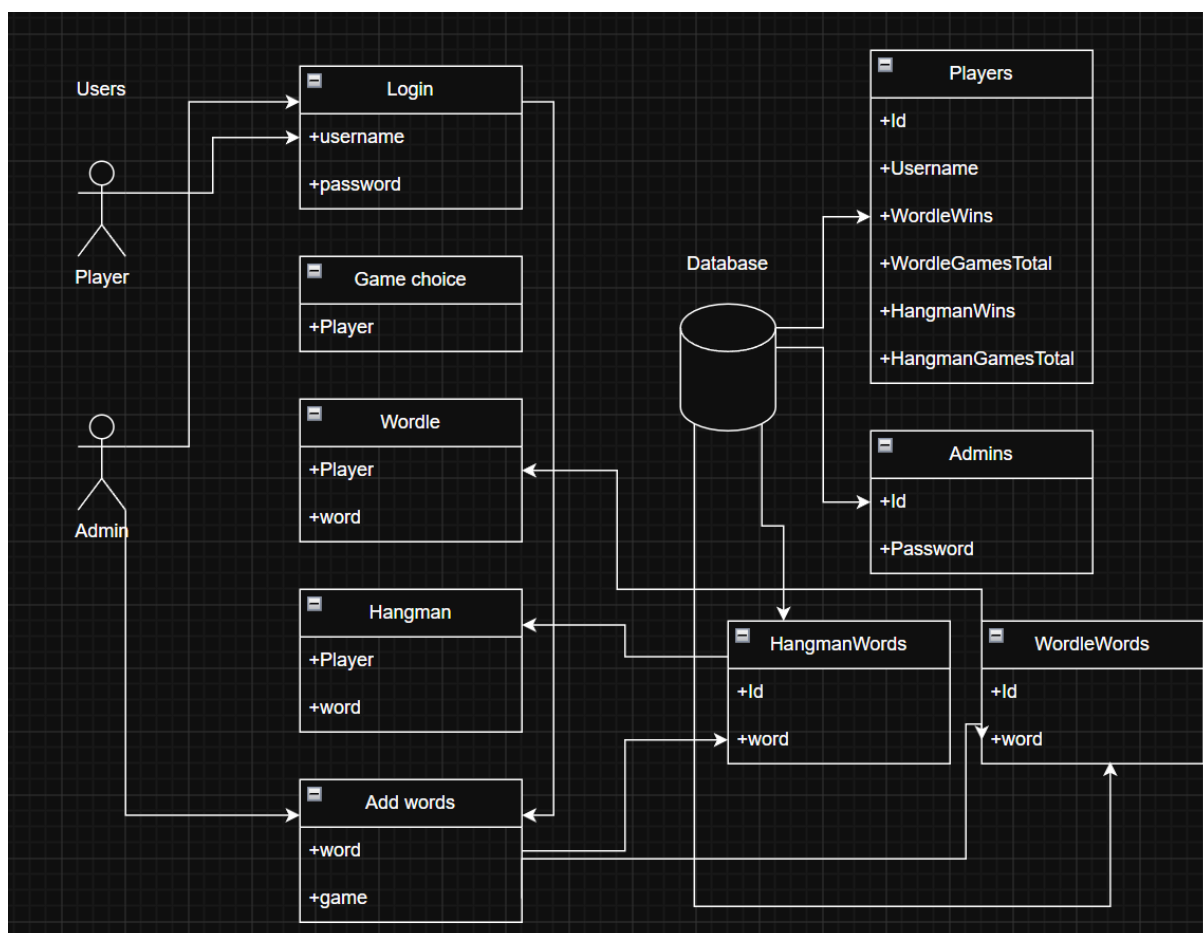
- 3.5.1. Потребителски интерфейс и виртуална клавиатура:** Тъй като традиционната Wordle игра изисква лесно въвеждане, в `Wordle.Designer.cs` е изградена пълна виртуална клавиатура с бутони за всяка буква от българската азбука (от `button1` до `button30`), бутон за изтриване (`buttonDeleteChar`) и бутон за потвърждение на думата (`buttonCheckWord`). Текстът се визуализира динамично в `textBox1`.
- 3.5.2. Инициализация и залагане на тайна дума:** При зареждане на формата (`Wordle_Load`), от базата данни се избира произволна дума, която играчът трябва да познае.
- 3.5.3. Следване на състоянието и статистика:** Играта следи текущия брой опити чрез `GuessCounter`. При победа или при загуба се актуализира статистика за общия брой изиграни игри на потребителя (`CurrentPlayer.PlayerTotalGames`) и броя победи (`CurrentPlayer.PlayerWordlWins`), след което се извършва навигация обратно към екрана за избор на игра (`FormChooseGame`).



#### 4. Взаимодействие потребител-приложение-база данни и UML-диаграма

Проектирането на десктоп приложението за игрите „Бесеница“ и „Wordle“ се базира на принципите на обектно-ориентираното програмиране (ООП) и трислойната софтуерна архитектура. За визуализиране на статичната структура на системата, класовете, техните атрибути, методи и връзки помежду им е изготвена UML диаграма на класовете.

Архитектурата на приложението е разделена логически и функционално на три основни компонента: **Users**, **Forms** и **Database**.



##### 4.1. Модел и управление на потребителите

Компонентът Users обхваща класовете, които отговарят за представянето на потребителския профил в системата, съхранението на текущото състояние на играча в паметта и управлението на неговата персонална статистика за двете игри.

Потребителите са разделени на два вида:

- Player - Това е централният обект в този слой. Той капсулира цялата информация за регистрирания играч, включващ атрибутите:
  - id (int): Уникален идентификатор на потребителя в базата данни.
  - username (varchar): Потребителско име, използвано за идентификация.
  - hangmanGamesTotal / wordleGamesTotal (int): Брой изиграни игри за съответната игра.

- hangmanWins / wordleWins (int): Брой спечелени игри.
- Admin - Този обект съдържа по-малко информация, защото неговата роля е да редактира възможните думи, които могат да участват в игрите Wordle и Hangman. Неговите атрибути са:
  - id (int): Уникален идентификатор на потребителя в базата данни.
  - password(varchar): Това е паролата, нужна за влизане като администратор, за да не може злонамерено лице да редактира думите.

#### 4.2. Графичен потребителски интерфейс - GUI

Компонентът Forms представлява презентационния слой на десктоп приложението. Той включва всички класове, които наследяват базовите интерфейсни прозорци на използваната графична библиотека. Тези класове отговарят за улавянето на потребителски събития (като натискане на бутони или клавиши) и визуализирането на игровия процес.

Потребителският интерфейс е съставен от 5 форми:

- Login - Това е първата форма, която се зарежда при стартиране на приложението. Нейната цел е да идентифицира потребителя и да защити личния му игрови профил, изисквайки име и парола.
- Game choice - Изпълнява ролята на централен навигационен панел (Hub) за приложението след успешен вход в системата. От там могат да се достъпят всички други форми.
- Wordle - Капсулира целия визуален интерфейс и събитийна логика за петбуквената игра с думи, като използва случайна дума от предназначените в базата данни.
- Hangman - Осигурява визуалната среда за класическата игра на познаване на думи чрез избиране на отделни букви, като отново използва случайна дума от предназначените в базата данни.
- Add words - Специализиран модул за управление на езиковата база данни на приложението, достъпен само от администратори, като добавя дума към речника на определената игра.

#### 4.3. Достъп до данни

Компонентът Database е фундаменталният слой (Data Access Layer), който отговаря за дългосрочното съхранение на информацията. Без него състоянието на приложението би се загубило след затваряне на десктоп прозореца. Този компонент управлява базата данни.

Той съдържа както информацията за потребителите (Players и Admins), така и думите, които могат да участват в игрите (WordleWords и HangmanWords).

## 5. Използвани алгоритми

В кода са внедрени няколко базови алгоритмични концепции за обработка на данни и управление на състоянията:

### 5.1. Текстово съпоставяне с регулярни изрази (Regex):

В Start.cs се използва шаблонен израз, като имплементираният алгоритъм проверява дали потребителското име започва с #A, последвано от цифра от 1 до 5, за да определи типа логващ се потребител и оттам да го класифицира като администратор и да му предостави съответните права описани в точка 4. Като следващ етап от верифицирането се прави заявка към контекста на базата данни, с цел проверка съществуването на такъв потребител.

- Използвайки регулярен израз за разпознаване на типа логващ се потребител се добавя ниво на сигурност по отношение на верификацията - няма индикатори за обикновения потребител за съществуващия администраторски режим.
- Базата данни се претърсва за точно съвпадение на предоставените входни данни използвайки LINQ заявка;

```
private void buttonLogIn_Click(object sender, EventArgs e)
{
    using (DbwordGamesContext context = new DbwordGamesContext())
    {
        Regex regex = new Regex("#A[1-5]");

        if (regex.IsMatch(textBoxUsernameInput.Text))
        {
            var admin = context.Admins.FirstOrDefault(
                a => a.AdminUsername == textBoxUsernameInput.Text
                && a.AdminPassword == textBoxUserPassword.Text);

            if (admin != null)
            {
                FormAddNewWords.CurrentAdmin = admin;
                FormAddNewWords form = new FormAddNewWords();
                form.Show();
                this.Hide();
            }
            else
            {
                MessageBox.Show("Няма такъв администратор!");
            }
        }
        else
        {
            var player = context.Players.FirstOrDefault(
                p => p.PlayerUsername == textBoxUsernameInput.Text
                && p.PlayerPassword == textBoxUserPassword.Text);

            if (player != null)
            {
                FormChooseGame.CurrentPlayer = player;
                FormChooseGame form = new FormChooseGame();
                form.Show();
                this.Hide();
            }
            else
            {
                MessageBox.Show("Няма такъв потребител!");
            }
        }
    }
}
```

## 5.2. Генериране на псевдослучайни индекси (Randomization):

В Hangman.cs се използва класът Random от System библиотеката като част от логиката на два алгоритъма, отделени в съответстващите им методи в контекста на partial class FormHangman:

- **Метод GenerateWord()** - генерира по случаен принцип думата, която потребителят трябва да познае в хода на играта. При отворена връзка с контекста на базата данни се избира дума, чийто индекс в базата съвпада с генерирания от програмата.
  - Ограниченията за интервала, в който се генерират индексите, е от 1 до броя на записите в context.WordsHangmen класа, т.к. номерацията на записите в базата е зададена като IDENTITY(1,1);
  - В LINQ заявката се търси първото точно съвпадение с .First(), а не с .FirstOrDefault(), т.к. програмата не може да приеме за изходна дума празен низ.

```
public void GenerateWord()
{
    using (DbwordGamesContext context = new DbwordGamesContext())
    {
        Random random = new Random();
        int index = random.Next(1, context.WordsHangmen.Count());

        Word = context.WordsHangmen.First(w => w.WordId == index);
    }
}
```

- **Метод LoadWord()** - генерира низ с празни позиции, от които играча трябва да се ориентира колко е дължината на думата. В рамките на същия метод се генерира и произволен индекс, като символа отговарящ на него трябва да се направи видим.
  - Ограниченията за интервала, в който се генерират индексите, е от 1 до броя на символите на Word.Word, т.к. думите записани в това свойство нямат постоянна дължина.

```
public void LoadWord()
{
    List<char> word = (new string('-', Word.Word.Length)).ToList();

    DisplayedWord = word;

    Random random = new Random();
    int revealedIndex = random.Next(0, Word.Word.Length);

    word[revealedIndex] = Word.Word[revealedIndex];

    textBoxWord.Text = string.Join(' ', word).ToString();
}
```

### 5.3. Линеино търсене (Linear Search):

В метода `GuessCharacterInWord(string ch)`, програмата итерира през символите на думата един по един, за да провери дали въведената от потребителя буква съществува в нея или не, като итерирането се спира при намиране на първо съвпадение.

```
1 reference
public bool GuessCharacterInWord(string ch)
{
    bool isContained = false;

    for (int i = 0; i < Word.Word.Length; i++)
    {
        if (Word.Word[i].Equals(ch))
        {
            isContained = true;
            break;
        }
    }
    return isContained;
}
```

### 5.4. Динамично адресиране на GUI елементи (Рефлексия):

Вместо масив от контролни елементи, кодът използва алгоритъм за намиране на правилния Label в Windows Forms чрез неговото текстово име:

```
this.Controls.Find($"w{GuessCounter}l{i + 1}", true)
```

Това позволява с формула динамично да се достъпват компоненти като `w1l1` (ред 1, буква 1) до `w6l5` (ред 6, буква 5) в зависимост от текущия ход и индекс.

### 5.5. Алгоритъм за оценка и оцветяване (Сравняване на низове):

Това е ядрото на играта Wordle. Алгоритъмът итерира през всяка позиция (от 0 до 4) на въведената дума и я сравнява с тайната дума (`Answer.Word`):

- *Точно съвпадение (Зелено)*: Ако буквата на позиция `i` във въведената дума съвпада напълно с буквата на същата позиция в тайната дума, фонът на клетката се оцветява в зелено (`Color.Green`).
- *Частично съвпадение (Жълто)*: Ако буквата не е на правилната позиция, но се съдържа някъде в тайната дума (`Answer.Word.Contains(...)`), клетката се оцветява в жълто (`Color.Yellow`).
- *Липса на съвпадение (Сиво)*: Ако буквата изобщо я няма в думата, цветът става сив (`Color.Gray`).

*Примерен код, демонстриращ промяната в състоянието на GUI елемент:*

```
if (w1l1.Text[0].Equals(Answer.Word[0]))
    w1l1.BackColor = Color.Green;
else if (Answer.Word.Contains(w1l1.Text))
    w1l1.BackColor = Color.Yellow;
else
    w1l1.BackColor = Color.Gray;
```

### 5.6. Управление на състояния (State Machine чрез Switch-case):

В метода `ChangePicture()` се превключва състоянието на играта спрямо стойността на `GuessCounter`, което определя кое изображение да се зареди и дали играта преминава в състояние "Загуба".

- Изображенията се зареждат от папката `Resources`, като те са номерирани по ред на появяване в хода на играта, с цел по-удобна справка.

```
public void ChangePicture()
{
    switch (GuessCounter)
    {
        case 5:
            pictureBoxHangman.Image = Properties.Resources.hangman2;
            break;
        case 4:
            pictureBoxHangman.Image = Properties.Resources.hangman3;
            break;
        case 3:
            pictureBoxHangman.Image = Properties.Resources.hangman4;
            break;
        case 2:
            pictureBoxHangman.Image = Properties.Resources.hangman5;
            break;
        case 1:
            pictureBoxHangman.Image = Properties.Resources.hangman6;
            break;
        case 0:
            pictureBoxHangman.Image = Properties.Resources.hangman7;

            Player.PlayerTotalGames++;
            MessageBox.Show($"Зазубихме! Думата беше {Word.Word}");

            FormChooseGame form = new FormChooseGame();
            form.Show();
            this.Hide();

            break;
    }
}
```

### 5.7. Алгоритъм за псевдослучаен избор на тайна дума:

В метода `GenerateAnswer()` в класа `Wordle`, този алгоритъм отговаря за инициализирането на играта чрез избиране на тайна петбуквена дума от външен текстов файл (`5LetterWords.txt`)

Процесът следва стъпките:

- Отваряне на поток за четене на данни от файла (`StreamReader`).
- Инициализиране на генератор на псевдослучайни числа (`Random rnd = new Random()`).
- Генериране на случайно цяло число в предварително зададен диапазон (от 0 до 65), което представлява номера на реда, на който се намира търсената дума.
- Линеино обхождане (сканиране): Използва се цикъл `for`, който итерира до генерираното число, като чете и игнорира редовете (`sw.ReadLine()`), докато достигне целевия ред.
- Прочитане и запазване на целевата дума в глобалната променлива `answer`



```
private void GenerateAnswer()
{
    using (var sw = new StreamReader("../.../5LetterWords.txt"))
    {
        Random rnd = new Random();
        int lineNumber = rnd.Next(0, 65);
        for (int i = 0; i < lineNumber; i++)
        {
            sw.ReadLine();
        }
        answer = sw.ReadLine();
    }
}
```

### 5.8. Алгоритъм за управление на състоянието на играта:

Намира се в края на метода button31\_Click. Този алгоритъм следи прогреса на потребителя и определя кога играта трябва да приключи.

- При всеки потвърден опит се инкрементира броячът за текущ ход (currentWord++).
- Условие за победа: Проверява се дали целият въведен низ съвпада напълно с тайната дума. При успех се увеличават променливите за победи и потребителят се връща в главното меню.
- Условие за загуба: Проверява се дали са изчерпани максималният брой опити (if (currentWord == 7)). Ако потребителят не е познал думата след 6-ия опит се отчита загуба (totalLoses++) и на екрана се разкрива тайната дума.

```
currentWord++;
if (textBox1.Text == answer)
{
    MessageBox.Show("Поздравления! Познахте думата!");
    totalWins++; winsInARow++;

    ChooseGame form = new ChooseGame();
    form.Show();
    this.Hide();
}
if (currentWord == 7)
{
    MessageBox.Show($"Вие загубихте! Думата беше {answer}");
    totalLoses++; winsInARow = 0;

    ChooseGame form = new ChooseGame();
    form.Show();
    this.Hide();
}
```

## 6. Приложени принципи на ООП

### 6.1. Капсулация (Encapsulation)

Класовете капсулират своите данни (свойства) и поведение (методи):

- *FormHangman* - следните данни са капсулирани в отделни свойства - Player, Word (скритата дума), DisplayedWord, GuessCounter, UsedChars, като достъпа до тях е направен public, за да може информацията, записана в тях да бъде наследена от друга форма, а също така да се избегне локално копие на данните за играча.

```
5 references
public static Player Player { get; set; } = new Player();
10 references
public static WordsHangman Word { get; set; } = new WordsHangman();
4 references
public static List<char> DisplayedWord { get; set; } = new List<char>();
5 references
public static int GuessCounter { get; set; }
2 references
public static List<char> UsedChars { get; set; } = new List<char>();
```

- *FormWordle* - обединява в общо тяло състоянието на играта (променливите GuessCounter, Word, Answer) и методите за манипулация на интерфейса. Полетата са защитени, а споделянето на данни (например текущия играч CurrentPlayer) става чрез статични свойства с контролиран достъп (getters и setters).

```
6 references
public static Player CurrentPlayer { get; set; } = new Player();
32 references
public static WordsWordl Word { get; set; } = new WordsWordl();
63 references
public static WordsWordl Answer { get; set; } = new WordsWordl();
9 references
public static int GuessCounter { get; set; }
```

### 6.2. Наследяване (Inheritance)

- Всички класове за форми (напр. public partial class FormStart : Form) - наследяват базовия клас Form от пространството от имена System.Windows.Forms.
  - Благодарение на това, създадените класове придобиват наготово всички характеристики на прозорец от графичния интерфейс.

```
public partial class Start : Form
public partial class FormHangman : Form
public partial class Wordle : Form
```

- Класовете *FormHangman* и *FormWordle* - наследяват данните за текущия потребител още със създаването на инстанцията на класа.

```
FormChooseGame.CurrentPlayer = player;
FormChooseGame form = new FormChooseGame();
```

- Класът *FormAdmin* - наследяват данните за текущия администратор още със създаването на инстанцията на класа.

```
FormAddNewWords.CurrentAdmin = admin;  
FormAddNewWords form = new FormAddNewWords();
```

### 6.3. Абстракция (Abstraction)

Взаимодействието с базата данни се осъществява чрез DbwordGamesContext, който е част от Entity Framework (ORM). Контекстът абстрахира сложните SQL заявки, релации и управление на връзката с базата данни. Програмистът работи с обекти от реалния свят (напр. context.WordsWordls.Add(word)), без да се интересува от конкретния SQL синтаксис, който се изпълнява "под капака".

### 6.4. Използване на partial класове

Класът Wordle е разделен на две части (partial):

- Wordle.cs – съдържа потребителския код, алгоритмите и бизнес логиката (събитията при натискане на бутони).
- Wordle.Designer.cs – съдържа автоматично генерирания код за подредбата на интерфейса.

Това е чиста форма на архитектурно разделение, което пречи на визуалните настройки да усложняват логическия код.

### 6.5. Роля на EF Core в проекта и връзка с ООП

В проекта, EF Core служи като технологичен мост между обектно-ориентирания C# код на Windows Forms приложението и релационната база данни (DbwordGamesContext), съхраняваща информацията за играчите, администраторите и думите. благодарение на нея се избягва писането на сурови SQL заявки (SELECT, INSERT, UPDATE), като вместо това се работи с чисти C# обекти и LINQ (Language Integrated Query) заявки.

Ключови EF Core компоненти, използвани в кода:

- **DbContext (DbwordGamesContext):** Това е централният клас в EF Core, който представлява сесията с базата данни. Той се грижи за конфигурирането на връзката, проследяването на промените (Change Tracking) и транзакциите.
- **DbSet<T> колекции:** Въпреки че точната дефиниция на контекста е в отделен файл, от кода се вижда наличието на колекциите context.Players, context.Admins, context.WordsWordls и context.WordsHangmen. Всеки DbSet съпоставя C# клас (модел) към конкретна таблица в базата данни.

## 7. Етапи на разработка на софтуерното приложение

Процесът по изграждане на десктоп приложението за игрите „Бесеница“ и „Wordle“ е реализиран чрез последователен и структуриран инженерен подход. Разработката следва класическия жизнен цикъл на софтуерния продукт и преминава през следните пет основни етапа:

### 7.1. Анализ на изискванията и планиране.

В тази начална фаза се извършва детайлно проучване на предметната област:

- Анализират се правилата и математическите/логическите модели на двете игри.
- Дефинират се основните функционални изисквания - например необходимостта от система за регистрация и вход, разделяне на потребителите на обикновени играчи и администратори, както и изискването за трайно съхранение на индивидуалната статистика на всеки играч.
- Избират се подходящите технологии за реализация - Visual Studio и SSMS, използвайки езиките C# и TSQL, както и Windows Forms

### 7.2. Софтуерно проектиране и моделиране

Този етап трансформира изискванията в конкретни технически планове.

- Архитектурен дизайн: Изготвя се UML диаграма на класовете, която разделя системата на три основни логически слоя - Слой на данните (Database), Бизнес слой и Презентационен слой (Forms).
- UI/UX дизайн: Проектира се графичният потребителски интерфейс (GUI) на петте основни форми (Login, Menu, Wordle, Hangman, Add Words). Целта е създаването на интуитивна визуална среда, включваща виртуални клавиатури и динамично оцветяване на елементите.

### 7.3. Програмна реализация (Implementation / Coding)

Това е същинската фаза на кодиране, в която проектираните модели се превръщат в работещ софтуер чрез езика C#.

- Програмират се алгоритмите за псевдослучаен избор на думи от файловата база.
- Имплементира се логиката за валидация на потребителския вход и алгоритмите за оценка на състоянието на играта (победа/загуба).
- Прилагат се принципите на обектно-ориентираното програмиране (наследяване, капсулиране и абстракция) за осигуряване на гъвкава и лесна за поддръжка на кода.

#### **7.4. Тестване и отстраняване на грешки (Testing and Debugging)**

След завършване на базовия код се провеждат серия от тестове за гарантиране на качеството на приложението.

- **Функционално тестване:** Проверява се дали всяка форма реагира правилно на потребителските събития (натискане на бутони, преминаване между екрани).
- **Обработка на изключения (Exception Handling):** Системата се тества за непредвидени ситуации (например опит за изтриване на буква от празно текстово поле) и се добавят try-catch блокове за предотвратяване на критични сригове.
- **Отстраняват се откритите логически грешки в алгоритмите за изчисляване на статистиката (победи и поредици от победи).**

#### **7.5. Внедряване и документиране**

В последния етап софтуерът се компилира в завършен изпълним файл (.exe), готов за стартиране в локална Windows среда. Успоредно с това се изготвя настоящата дипломна работа, която служи като пълна техническа и потребителска документация на проекта, описваща архитектурата, алгоритмите и начина на експлоатация на създаденото приложение.

## 8. Използвани технологии

За реализирането на десктоп приложението и изпълнението на всички заложи изисквания за производителност, сигурност и удобство, бяха подбрани и използвани следните съвременни софтуерни технологии и инструменти:

**C# (C-Sharp):** Основен обектно-ориентиран език за програмиране, използван за написването на цялостната бизнес логика, алгоритмите на игрите и управлението на състоянията на приложението.

**Windows Forms:** Графична рамка (Framework) на Microsoft, базирана на .NET, която беше използвана за проектиране и изграждане на графичния потребителски интерфейс (GUI) - форми, бутони, текстови полета и виртуални клавиатури.

**Microsoft SQL Server & T-SQL:** Система за управление на релационни бази данни (RDBMS), използвана за сигурно и трайно съхранение на профилите, паролите, статистиките и речниците с думи. Чрез T-SQL са реализирани съхранените процедури за манипулация на данните.

**Entity Framework Core:** Съвременен обектно-релационен мапер (ORM), който улеснява комуникацията между C# приложението и SQL базата данни. Чрез него таблиците са представени като C# класове (Entities), което позволява използването на обектно-ориентиран подход при работа с данните.

**LINQ (Language Integrated Query):** Технология, интегрирана в C#, която беше използвана за писане на кратки, четими и сигурни заявки към базата данни директно в кода (например при търсене на конкретен потребител или избиране на произволна дума).

**Visual Studio (IDE):** Основната интегрирана среда за разработка, в която е написан, компилиран и тестван програмният код на приложението.

**SQL Server Management Studio (SSMS):** Софтуерен инструмент, използван за визуално администриране на базата данни, създаване на таблиците и тестване на SQL заявките.

## 9. Роли на всеки от екипа

### 9.1. Мирослава Венелинова:

**База данни и съхранение на данни:** Проектиране на релационната база данни (SQL Server), създаване на таблици за думи, играчи и администратори, както и интегрирането на базата данни с приложението (Data Access Layer).

**Разработване на игралния модул "Бесеница" (Hangman.cs):** визуално изчертаване, алгоритми за познаване на букви и отчитане на грешки.

**Създаване на екрана за вход (Start.cs):** логика за разпознаване на роли.

**Изграждане на администраторския панел (Admin.cs):** Използва се за добавяне и управление на речниците с думи.

**Интеграция на компонентите:** Свързване на всички форми в единна работеща система, реализиране на преходите между различните екрани (прехвърляне на фокуса, скриване/показване на прозорци) и управление на потока на данните между тях.

**Проектна документация:** Съавторство при съставянето, структурирането и написването на обяснителните текстове и анализи в настоящата документация.

### 9.2 Радослав Цветков:

**Игрален модул "Wordle" (Wordle.cs):** Цялостно проектиране и програмна реализация на формата за играта "Wordle". Това включва алгоритмичната логика за проверка на въведените думи, динамичното оцветяване на предположените букви и изграждането на виртуалната клавиатура.

**Главно меню и навигация (ChooseGame.cs):** Създаване на навигационната форма за избор на игра и реализиране на логиката за извличане и визуализиране на текущата потребителска статистика (брой победи, изиграни игри).

**Архитектурно моделиране:** Изготвяне на концептуалната архитектура на приложението и чертане на UML диаграмата на класовете, визуализираща връзките между слоевете Users, Forms и Database.

**Проектна документация:** Съавторство при съставянето, структурирането и форматирането на настоящата документация (описание на архитектурата, алгоритмите и интерфейса).

## ЗАКЛЮЧЕНИЕ

Разработването и успешната реализация на десктоп приложението за игрите с думи „Бесеница“ и „Wordle“ доказват, че съвременните софтуерни технологии могат да служат като ефективен мост между развлечението и образованието. В ерата на масовата дигитализация, в която технологиите често се свързват с безмисловното потребление от младото поколение, този проект предлага алтернатива - интерактивна и интелектуална среда, насочена към развиване на когнитивните умения, логическото мислене и езиковата грамотност.

В хода на работата бяха изпълнени всички предварително заложи цели чрез създаването на стабилно трислойно десктоп приложение на C# (Windows Forms), интегрирано с релационна база данни (SQL Server и EF Core) за сигурно съхранение на профили, речници и статистика. Разработеният софтуер се отличава с интуитивен интерфейс, подходящ за потребители от всички възрасти, и включва функционален администраторски панел за лесно управление и разширяване на езиковата база.

Като ключов извод от съвместната работа на екипа може да се посочи, че софтуерното инженерство изисква не само технически умения за писане на код, но и задълбочено планиране, проектиране на архитектурата (чрез UML моделиране) и правилно разпределение на задачите. Проектът успешно съчетава сложна backend логика и бази данни с динамичен презентационен слой на потребителския интерфейс.



### Източници на информация:

- [1] Microsoft SQL documentation (2026 г.) Microsoft: <https://learn.microsoft.com/en-us/sql/?view=sql-server-ver17> - посетено на април 2026 г.
- [2] Unified modeling language UML introduction (16.04.2026 г.) GeeksForGeeks: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/> - посетено април 2026 г.
- [3] Entity Framework documentation hub (2026 г.) Microsoft: <https://learn.microsoft.com/en-us/ef/> - посетено на април 2026 г.
- [4] Какви полезни умения развиват у децата настолните игри? (29.11.2018 г.) Детска планета: <https://detskaplaneta.com/blog/nastolnite-igri-kakvi-polezni-umeniya-razvivat-u-decat> - посетено на 16.05.2026 г.
- [5] Кои са класическите настолни игри и какви умения развиват при децата ни? (21.02.2026 г.) Игри за здравето: [https://toysforhealth.com/blog?journal\\_blog\\_post\\_id=13](https://toysforhealth.com/blog?journal_blog_post_id=13) - посетено на 16.05.2026 г.